CareerLens - Viva Notes

> **How this document was prepared (2026-10-17):** every statement below is based on a direct read of this repository's files (`src/`, `backend/`, config files, tests, `dataset/`, `research/`). Exact file paths are given so you can show the code during the viva. Anything that could not be verified from the code is flagged with ⚠. No application code was modified, and no `.env` secrets were read or are exposed here.

---

1. Project overview and architecture

What CareerLens is: a full-stack web app that helps a job seeker go from a PDF resume to interview-ready. A user uploads a resume, the app parses it, scores it against a job description (ATS-style), analyses skill gaps, recommends courses, generates tailored interview questions, and runs a proctored 3-round mock interview with scoring.

Architecture (3 layers, one repository):

Layer	Where	What it does
1. Presentation	`src/` (React SPA)	Pages, routing (`src/routes/index.tsx`), UI components, client-side stores
2. API / service	`backend/server.js` (Express)	~13 REST endpoints under `/api/v1`, validation, error handling
3. Intelligence	`backend/aiProvider.js`, `backend/llm.js`, `backend/skillGapLlm.js`, `backend/mockInterviewLlm.js`, `backend/ruleBasedFallback.js`, `backend/questionBank.js`, `backend/pdfExtract.js`	PDF/OCR extraction, AI provider routing (Gemini → Groq), deterministic fallbacks

- One command runs everything: `npm run dev` starts Vite (web, port **5173**, strict) and `node backend/server.js` (API, port **5000**, `PORT` env) together via `concurrently` (root `package.json`).
- Example request flow (match analysis): UI uploads PDF → `POST /api/v1/resume/parse` → `pdfExtract.js` (text or OCR) → deterministic parser → `POST /api/v1/match/analyze` → resume authenticity check → `analyzeMatch()` → AI (Gemini, else Groq) **or** the rule-based engine → JSON response → UI renders score, skills, gaps, keywords.

- **Design principle visible throughout the code:** every AI feature has a deterministic fallback, and every result carries a truthful `source` label ("AI-generated (Gemini)" / "AI-generated (Groq fallback)" / "Rule-based fallback") rendered by `src/components/ui/result-source.tsx`.

2. Frontend and backend technologies

Frontend (`package.json`, `vite.config.ts`, `src/main.tsx`):

- React **19.2**, TypeScript **~6.0**, Vite **8**, `react-router-dom` **7** (all routes lazy-loaded).
- Tailwind CSS **v4** via `@tailwindcss/vite`; design tokens as CSS variables in `src/index.css`.
- `framer-motion` 13 (animation), `lucide-react` (icons), `clsx` + `tailwind-merge` (`cn()` in `src/lib/utils.ts`).
- ⚠ `recharts` is declared in `package.json` but no import was found anywhere in `src/` (unused).
- State management: **no Redux/Zustand**. Three small module-level stores with subscriber sets (`src/lib/resume-store.ts`, `saved-jobs-store.ts`, `compare-store.ts`) + React Context for theme (`src/lib/theme.tsx`) and the global resume-upload modal. `src/lib/run-guard.ts` dedupes requests and discards stale responses (e.g. when a new resume replaces an old one).
- Frontend test tooling: Vitest 5 + jsdom + Testing Library; linter `oxlint`.

Backend (`backend/package.json`):

- Node.js `^22.22.2 || ^24.15.0 || >=26.0.0`, **Express 4**, `cors`, `multer` (memory storage), `dotenv`.
- PDF text: `pdf-parse`. OCR for scanned PDFs: `tesseract.js` (WASM; English language data **vendored** in `backend/tessdata/` - no CDN needed).
- Gemini SDK: `@google/genai`. Groq is called over plain HTTPS `fetch` (OpenAI-compatible endpoint).
- ⚠ `zod` is declared in `backend/package.json` but no import was found in any backend source `.js` file (unused - schemas are hand-validated in `aiProvider.js`).
- Tests: built-in `node:test` runner (`npm --prefix backend test`).

3. Database and dataset

There is no database. Verified: no `*.db` / `*.sqlite` files anywhere, no DB dependency. Persistence actually used:

- **In-memory (server):** mock-interview sessions (`backend/mockInterviewStore.js` - a `Map`); AI result cache, in-flight dedupe, provider cooldowns (`backend/aiProvider.js`); uploaded files (`multer.memoryStorage()`) - RAM only. All of it is lost on restart.
- **Browser localStorage (client):** theme, saved jobs, compare list, profile fields (`careerlens.user_name`, `user_email`, `user_phone`, `user_city`, `user_title`,

`user_bio`), the skill-gap→career-growth handoff (`careerlens.skillgaps`), and demo credentials (`careerlens.pass_<email>`, `careerlens.name_<email>` - see section 7).

- **The resume itself is deliberately session-only:** it is not persisted; `src/main.tsx` runs `clearPersistedResumeData()` before the app renders, and `src/lib/resume-store.ts` keeps it in memory only.

Job listings: 14 curated static listings in `src/data/jobs.ts` (12 employers, some with real apply URLs in `redirectUrl`). There is no live job-board API. `src/data/matches.ts` is an **empty array** (pre-computed matches removed).

`dataset/` folder: a standalone **synthetic** resume-job matching dataset (60 rows, 6 fictional junior-role families × 10 synthetic candidate profiles), with deterministic skill-overlap scores and rule-derived labels, plus `validate_dataset.py`, data dictionary and README. It is explicitly **not** exported from CareerLens, not real user data and not human-validated.

`research/` folder: offline research artifacts - frozen synthetic benchmark (`research/data/benchmark-v1/benchmark_v1.json`, 120 pairs), pilot dataset, call ledger (`research/logs/CALL_LEDGER.csv`), RQ1 agreement results (Pearson r 0.909, n=31 - agreement with a rule-based score, not accuracy), manuscript drafts and analysis scripts (`research/scripts/`). ⚠ Several files referenced by `research/results/reproducibility/SHA256SUMS.txt` (e.g. `RESEARCH_PROTOCOL.md`, benchmark data dictionaries) are missing from this checkout.

4. AI/LLM providers, exact models, APIs, prompts, integrations

Provider layer: `backend/aiProvider.js`.

- **Primary provider: Google Gemini** via the `@google/genai` SDK. Default model: `gemini-flash-latest` (override with `GEMINI_MODEL`).
- **Fallback provider: Groq** via HTTPS `POST https://api.groq.com/openai/v1/chat/completions`. Default model: `openai/gpt-oss-120b` (override with `GROQ_MODEL`).
- Strict structured output mode is only used for `openai/gpt-oss-20b`, `openai/gpt-oss-120b`, `qwen/qwen3.8-27b`; any other Groq model gets best-effort `json_object` mode with the JSON Schema embedded in the system prompt (`STRICT_OUTPUT_MODELS` in `aiProvider.js`).
- **Selection:** `AI_PROVIDER = gemini | groq | none` (override), else auto: Gemini if `GEMINI_API_KEY` exists, else Groq if `GROQ_API_KEY` exists, else **no AI at all** (keyless rule-based mode). `AI_PROVIDER=none` forces keyless mode.
- **Fallback rules:** the fallback is tried **exactly once**, only for eligible failures (HTTP 429, timeout, 5xx/network, invalid/empty JSON, schema mismatch) plus primary-side rejections (bad/missing key, HTTP 400/404). Credential/config errors never turn into a silent fake result. After a 429 with `retry-after`, the provider is put on cooldown; transient failures wait a jittered 200-1500 ms backoff before the fallback attempt.

- **Resilience extras:** max 4 concurrent provider calls (`AI_MAX_CONCURRENT_PROVIDER_CALLS`), in-flight dedupe via a SHA-256 hash of the full request, 5-minute in-memory result cache (100 entries, `AI_RESULT_CACHE_TTL_MS`), and error taxonomy (`AI_RATE_LIMITED`, `AI_TIMEOUT`, `AI_UPSTREAM_ERROR`, `AI_UNREACHABLE`, `AI_INVALID_JSON`, `AI_EMPTY_RESPONSE`, `AI_SCHEMA_MISMATCH`, `AI_UNAUTHORIZED`, `AI_NOT_CONFIGURED`, `AI_BAD_REQUEST`, `AI_PROVIDERS_EXHAUSTED`).
- **Structured output:** every task defines an OpenAPI-style JSON Schema (e.g. `matchAnalysisSchema` in `backend/llm.js`); responses are fence-stripped, parsed and normalized to the schema. Temperature: 0.1 for match/questions/evaluation, 0.2 for skill-gap/career/mock tasks.
- **Prompts** (system instructions, exact files):
  - Match analysis (`llm.js`): "Senior Technical Recruiter and ATS system" - hard skills 40%, experience relevance 30%, keywords/formatting 20%, soft skills 10%.
  - Interview questions (`llm.js`): "Senior Technical Interviewer" - exact resume/role split, ≤35-word questions, 3-4 bullet answer outlines.
  - Answer evaluation (`llm.js`): "Tech Interview Evaluator" - 0-100 score, ≤150-word feedback.
  - Skill gaps (`skillGapLlm.js`): "Career Development Coach and Technical Recruiter".
  - Career plan (`skillGapLlm.js`): "Senior Learning & Development Director" - ≤6 real courses, 3 milestones.
  - Mock interview (`mockInterviewLlm.js`): "expert technical interviewer and logic puzzle designer"; "Senior Technical and HR Evaluator" (0-10); "Head of Engineering" (scorecard).
- **Token/character budgets** are enforced per task (e.g. match 2000 tokens, answer eval 800, question generator 400 + 320×n; resume inputs clipped at 14,000 chars for match analysis).
- **Secrets:** only in `backend/.env` (loaded by `backend/loadEnv.js` relative to the backend folder). Variable *names*: `GEMINI_API_KEY`, `GEMINI_MODEL`, `GROQ_API_KEY`, `GROQ_MODEL`, `AI_PROVIDER`, `AI_FALLBACK_PROVIDER`, plus optional tuning. Never hardcoded, never logged.

5. Resume parsing, ATS scoring, skill matching, recommendations

5.1 PDF → text (`backend/pdfExtract.js`)

Text-layer PDFs are read with `pdf-parse` (20 s timeout). If too little text (scanned/photo PDF), embedded page images are pulled from the raw PDF bytes (JPEG `DCTDecode`, and `FlateDecode` images re-encoded as PNG) and read with **tesseract.js** OCR. Everything is bounded: 45 s total OCR budget, 30 s per image,

max 6 images, 8-megapixel downscale cap; one OCR worker, serialized and torn down after the request. Typed user-facing errors: encrypted / corrupt / timeout / no readable text. Minimum 10 words.

5.2 Resume parsing - deliberately **not** AI (`backend/llm.js`, `extractResumeDetails`)

Regex + section parser: name, email, phone, LinkedIn, location, headline; sections split by heading keywords (`SECTION_KEYWORDS`); skills, work experience (date ranges incl. "Present"), education (degree regexes, graduation year = last year on the line), projects, certifications, summary. Unicode is normalized first (ligatures, dashes, bullets). Because it is deterministic, an upload never spends AI tokens and never invents fields - missing fields stay missing.

5.3 Authenticity checks (same file)

Flags: text < 150 chars; placeholder names (`John/Jane Doe`, `[insert ...]`); lorem ipsum or unrelated text (no email/phone/degree/section signals); extreme keyword stuffing (same non-stopword ≥ 40 times); impossible timeline (end year < start year). Result: `isAuthentic` + `validationErrors`. A failed resume returns `atsScore 0` with the errors instead of being analysed (`server.js`, `/match/analyze`).

5.4 ATS scoring

- **AI path:** the prompt asks for the 40/30/20/10 weighting above and returns `atsScore` 0-100 plus `breakdown { skillsMatchScore, experienceScore, educationScore, keywordDensityScore }`, summary, matching/missing skills, gaps, keyword frequency. ⚠ Minor inconsistency to know: the prompt names *soft skills* as the 10% component, while the response schema (and rule engine) use *education* as the 10% component.
- **Rule-based path** (`backend/ruleBasedFallback.js`, `buildMatchAnalysis`): `atsScore = round(0.4·skills + 0.3·experience + 0.2·keyword + 0.1·education)` where skills = % of job requirements evidenced in the resume (50 if none found); experience = 0 if no dated entries, else `round(100·(0.4·min(1, entries/2) + 0.6·domainOverlap))`; keyword = mean per-requirement frequency band (0→0, 1-2→80, 3-8→100, ≥9→60 "overused"; 50 if none); education = 100/30 when the JD mentions education (degree present or not), 75 neutral when it does not.
- ⚠ **Do not confuse this with the jobs-page "Career Fit"**, which is a separate client-side formula in `src/lib/resume-analysis.ts`: hard 50% + soft 20% + experience 20% + education 10%; hard = 75% job-skill coverage + 25% resume relevance; levels ≥65 strong, ≥48 moderate, <48 weak.

5.5 Skill matching (`ruleBasedFallback.js`)

A curated vocabulary of ~115 skills with aliases and categories (SQL, React, Python, Docker, communication, ...) is scanned with word-boundary regexes; job requirements are vocabulary items found in the JD, ordered by first appearance,

priority "High" when on a requirement-style line ("must have", "essential", ...) or mentioned ≥2 times. Matched = requirements evidenced in the resume; missing = the rest; keyword statuses Missing / Present / Optimal / Overused.

5.6 Skill gaps (`POST /api/v1/skill-gaps/analyze`)

Required skills come from the JD, or from a curated role→skill map when the JD has none (`ROLE_SKILLS_MAP`, 9 roles). `readinessScore = round(100 × matched ÷ required)` (max 10 skills). Missing skills get category, priority, `currentProficiency: "None"`, a required level and a one-line description.

5.7 Recommendations

- **Career growth plan** (`POST /api/v1/career-growth/plan`): AI suggests ≤6 real courses with validated URLs (bad URL → Google search link) + 3 milestones. The rule-based alternative uses a curated `COURSE_DATABASE` (10 skills × 2 courses each) and a fixed 3-phase roadmap template.
- **Interview prep** (match page) produces questions + sample answer outlines + evaluation criteria from AI or the curated question bank (`backend/questionBank.js`, ~15 categories: Resume Foundations, Project Depth, Python, React, SQL, ML, Security, ...; deterministic selection scoring = anchor base + prominence + difficulty + diversity − repeats, near-duplicate questions (Jaccard ≥ 0.85) skipped, sha256 tiebreak).
- **Answer evaluation**: AI 0-100 or a deterministic 4×25 rubric (relevance to question keywords, evidence markers such as numbers/percentages, technical coverage, completeness word bands) with one improvement tip for the weakest dimension.

6. Mock interview workflow and evaluation

Setup (`src/pages/mock-interview.tsx`): user enters name/role, picks a curated job (or custom JD) and uploads a resume; `POST /api/v1/mock-interview/start` creates a server session (`interview-session-XXXX`, in memory) with proctor config (max 3 violations; webcam + mic + fullscreen required).

Three rounds (total 11 questions; durations in the start response):

1. **Aptitude** - 5 multiple-choice questions, 15 min.
2. **Technical** - 3 free-text questions, 25 min.
3. **HR & Behavioral** - 3 free-text questions, 15 min.

Question generation: AI ("3-round" prompt in `mockInterviewLlm.js`) or the deterministic generator (`buildMockQuestions`): aptitude MCQs from a fixed bank; technical questions built from the resume's own projects/skills and the JD's requirements; behavioural questions from fixed templates. `GET /mock-interview/questions` **strips** `correctAnswerIndex` from MCQs before sending them to the browser - answers stay server-side.

Grading: Round 1 MCQs are graded deterministically on the server (10 or 0, exact text/index comparison). Rounds 2-3 are graded by AI (0.0-10.0 + `technicalAccuracy`, `clarity`, `relevanceToRole`) or by the deterministic rubric

(`score = 0.4·technicalAccuracy + 0.3·clarity + 0.3·relevance`, 1-10). Every answer records its `source`.

Proctoring (browser side, verified in code): TensorFlow.js, **BlazeFace** (face + landmarks) and **CocoSSD** (object detection) are loaded from the jsDelivr CDN; webcam at 640×480. Face/gaze: `gazeRatio = |nose.x − leftEye.x| / |nose.x − rightEye.x|`; outside **[0.58, 1.7]** or no face detected → `GAZE_AWAY`. CocoSSD class `'cell phone'` with score **> 0.5** → `PHONE_DETECTED`. Tab switches → `TAB_SWITCH`. Violations are posted to `POST /mock-interview/proctor-event`.

Violations: stored per session; after the 3rd violation (`maxAllowedViolations: 3`, `backend/mockInterviewStore.js`) the session status becomes `TERMINATED`; the UI switches to a terminated screen, and a terminated session always scores 0.

Final scorecard (`POST /mock-interview/complete`, `buildFinalScorecard` or AI): per-round breakdown % = round avg ÷ (answered × 10); `overallScore` = mean of all answer scores × 10 (0-100; 0 if nothing answered). Hiring bands: **≥80 Strong Hire, ≥65 Recommended, ≥50 Borderline, else Not Recommended**. Up to 4 strengths and 4 improvement areas.

⚠ **Speech-to-text is simulated**: the mic button in `mock-interview.tsx` inserts a fixed sample paragraph, and `POST /mock-interview/transcribe-audio` returns a hardcoded transcript; the UI never calls that endpoint. Do not claim real audio transcription.

7. Authentication, security, testing, limitations

Authentication - ⚠ demo-level only (be honest in the viva)

- There is **no server-side authentication** and **no route guards**: any visitor can open `/dashboard` directly (`src/routes/index.tsx`), and no endpoint requires a login. The backend has no `/auth` endpoint (`backend/server.js`).
- `src/pages/login.tsx` is cosmetic: email format + password ≥ 4 chars. On signup it stores the **password in plaintext** in localStorage (`careerlens.pass_<email>`); on sign-in it only compares the password **if that key exists** - otherwise any ≥4-char password "signs in". "Explore Demo Account" just sets `careerlens.user_name = 'Guest'`.
- `src/lib/api-service.ts` never sends an `Authorization` header or tokens.

Security measures that DO exist

- CORS allow-list (`backend/server.js`): any localhost origin in development; in production only explicit `CORS_ORIGINS`.
- Upload limits: PDF-only, 5 MB (in-memory multer); audio 10 MB; JSON body 10 MB.
- MCQ correct answers hidden from the client; termination enforced server-side.
- Secrets live only in `backend/.env` (never committed, loaded via `backend/loadEnv.js` by path); logs contain only provider codes/durations,

never prompts, answers or payloads; API errors expose sanitized AI error codes only.

- `scripts/package-zip.ps1` builds a submission ZIP and excludes `.env` files (keeps `.env.example`), key material, logs, `node_modules`, `dist/`, and then checks the archive for leaks. ⚠ It requires a root `README.md` that is currently **missing**, so packaging would fail until that file exists.

Testing

- **Frontend** (Vitest + jsdom + Testing Library, `vitest.config.ts`): 17 suites under `src/`, including `src/lib/api-service.test.ts` (error classification), `src/lib/run-guard.test.ts`, store tests, `src/pages/jobs.test.tsx`, `src/pages/compare.test.tsx`, `src/pages/resume-lifecycle.test.tsx` (resume replacement + source labels) and `src/pages/match-interview-fallback.test.tsx`.
- **Backend** (`node:test`): 13 suites in `backend/tests/` - provider fallback matrix (`providerFallback.test.mjs`), cooldowns, CORS dev/prod, PDF/OCR (`pdfExtract.test.mjs`), question bank, rule-based formulas (`ruleBasedFallback.test.mjs`), real `Resume/` PDFs (`resumeFiles.test.mjs`), and an end-to-end feature flow. All AI calls are stubbed via a `globalThis.fetch` stub - **no live API calls in tests**.
- Commands: `npm test` (backend `node --test` then Vitest), `npm run lint` (oxlint), `npm run build` (`tsc -b && vite build`).

Limitations (state these honestly)

1. No real authentication/authorization; plaintext demo passwords; session IDs are 4-digit (`interview-session-XXXX`) and session endpoints are unauthenticated.
2. No database/persistence - restart loses all server state; single-process only.
3. Speech-to-text is simulated; the audio endpoint is a stub.
4. Proctoring is client-side heuristics (bypassable, needs camera + internet + CDN); it is a demo aid, not exam-grade security.
5. AI output quality varies; rule-based fallbacks approximate ATS/recruiter judgement and must not be treated as truth. The dataset is synthetic and small.
6. Demo/static UI areas: history cards and notifications are hardcoded samples; `matches.ts` is empty; `resume.tsx` and `career-profile.tsx` exist but are not routed; some dashboard components are unused; `recharts` and `zod` are declared but unused.
7. ⚠ AI prompt says soft skills = 10% while the response schema/rule engine use education = 10% (documented inconsistency, not a bug you need to hide).
8. ⚠ Research folder references some files missing from this checkout (`SHA256SUMS.txt`, protocol/README documents).

---

8. Thirty likely viva questions with short answers

Q1. What is CareerLens and what problem does it solve? A web app that helps job seekers prepare for applications: upload a PDF resume → ATS-style match score vs a job description → skill-gap analysis → courses/career plan → tailored interview questions → a proctored 3-round mock interview with a final scorecard. It targets students and early-career candidates.

Q2. Explain the architecture. Three layers: a React SPA (`src/`), an Express REST API (`backend/server.js`), and an intelligence layer (`backend/aiProvider.js` + task modules + deterministic fallbacks). `npm run dev` runs the web app (port 5173) and API (port 5000) together.

Q3. How do the frontend and backend communicate? Over HTTP JSON. The base URL is `http://127.0.0.1:5000/api/v1` by default (`src/lib/api-config.ts`), overridable with `VITE_API_BASE_URL`. `src/lib/api-service.ts` wraps all calls with timeouts (130 s default), one retry for connection errors, and typed errors for backend-down vs AI-provider failure.

Q4. What frontend technologies are used? React 19, TypeScript 6, Vite 8, React Router 7 (lazy routes), Tailwind CSS 4, framer-motion, lucide-react. State is handled by three small custom stores plus React Context - no Redux/Zustand.

Q5. What backend technologies are used? Node.js (≥22.22), Express 4, cors, multer (in-memory uploads), dotenv, pdf-parse for PDF text, tesseract.js for OCR, and `@google/genai` for Gemini. Groq is called with plain HTTPS fetch.

Q6. Which database does the project use? None - there is no DB or ORM. Mock-interview sessions live in an in-memory `Map` (`backend/mockInterviewStore.js`), uploads are held in RAM, and the AI cache is also in memory. All server state is lost on restart.

Q7. What is stored in the browser and how long does it live? localStorage: theme, saved jobs, compare list, profile fields, and the skill-gap handoff key. The resume itself is session-only - `src/main.tsx` clears persisted resume data at startup and no code path reads a resume from storage.

Q8. Where do the job listings come from? 14 curated static listings in `src/data/jobs.ts` (12 employers, some with real apply URLs). There is no live job API.

Q9. What is in the `dataset/` folder? A standalone synthetic resume-job matching dataset: 60 rows, 6 fictional job families × 10 synthetic candidate profiles, with deterministic skill-overlap scores, a data dictionary and a Python validator. It is explicitly not CareerLens user data and not human-validated.

Q10. Which AI providers and exact models are integrated? Primary: Google Gemini via `@google/genai`, default model `gemini-flash-latest`. Fallback: Groq via `https://api.groq.com/openai/v1/chat/completions`, default model `openai/gpt-oss-120b` (strict JSON mode for the gpt-oss and qwen3.8 models, else best-effort JSON). Both are configurable via `GEMINI_MODEL` / `GROQ_MODEL`.

Q11. How does the provider fallback work? The primary is tried first; on an eligible failure (rate limit, timeout, 5xx, network, invalid JSON, schema mismatch, or primary-only key/400 errors) the fallback is tried exactly once, after a jittered

200-1500 ms backoff. Rate-limited providers get a cooldown. There is in-flight dedupe, a 5-minute result cache and a 4-call concurrency cap (`backend/aiProvider.js`).

Q12. What happens when AI is unavailable? The deterministic rule-based engine (`backend/ruleBasedFallback.js`) computes real results from the actual resume/JD text (documented formulas), and the result is labelled "Rule-based fallback" so it is never passed off as AI. Credential/config errors instead surface as clear error codes - they never silently become rule results.

Q13. How is structured AI output guaranteed? Every task defines an OpenAPI-style JSON Schema; Gemini gets it as `responseSchema`, Groq as `json_schema` (or a prompt-embedded schema), and the response is fence-stripped, parsed and normalized to the schema with safe defaults. Bad JSON raises `AI_INVALID_JSON`, not garbage data.

Q14. How is a PDF resume converted to text? `pdf-parse` first (20 s timeout). If too little text, embedded page images are extracted straight from the PDF bytes and read with tesseract.js OCR (vendored English data; 45 s budget, ≤6 images, 8 MP cap). Errors are typed as encrypted / corrupt / timeout / no-text.

Q15. Why is resume parsing not done by AI? To make uploads deterministic, free of provider rate limits, and incapable of inventing details: `extractResumeDetails` (`backend/llm.js`) uses regexes and section heuristics, and missing fields stay missing. AI is reserved for higher-value tasks like match analysis and questions.

Q16. How does the "fake resume" authenticity check work? Deterministic checks: minimum length, placeholder names/brackets, lorem ipsum or unrelated text, extreme keyword stuffing (a non-stopword repeated ≥40 times) and impossible timelines. A failed resume returns `atsScore 0` with the reasons instead of being analysed.

Q17. How is the ATS score calculated? Rule engine: `round(0.4·skills + 0.3·experience + 0.2·keyword + 0.1·education)` with documented formulas for each component (skills = % of JD requirements evidenced; keyword bands 0/80/100/60; education 100/30 or 75 neutral). The AI path is asked to weight hard skills 40%, experience 30%, keywords 20%, soft skills 10%.

Q18. What is the difference between the ATS score and the jobs-page Career Fit score? They are different features: the ATS score comes from the backend match analysis; "Career Fit" is a client-side heuristic in `src/lib/resume-analysis.ts` (hard 50% + soft 20% + experience 20% + education 10%) used to rank the static job listings without any API call.

Q19. How are matching and missing skills determined? A curated vocabulary of ~115 skills with aliases is scanned in the resume and JD with word-boundary matching; JD items become requirements (priority High if on a "must have"-style line or repeated ≥2 times); matched = evidenced in the resume, missing = the rest, with keyword frequency statuses.

Q20. How does the skill-gap readiness score work? `readinessScore = round(100 × matched ÷ required)`, where required skills come from the JD or a curated role map, and matching considers both parsed resume skills and the

vocabulary scan. Missing skills get category, priority and "currentProficiency: None".

Q21. How are courses and the career plan generated? The AI path proposes ≤6 real courses (URLs validated, fallback to a search link) plus 3 phases. Without AI, a curated course database (10 skills × 2 courses) and a fixed 3-phase template are used. Every result is source-labelled.

Q22. How are interview questions generated on the match page? The AI returns exactly N questions split between resume-based and role-based with outlines and criteria; a contract filter keeps only valid items, and if a set ends up empty the curated question bank answers instead (deterministic scoring: anchor base + prominence + difficulty + diversity, near-duplicates skipped).

Q23. How are answers evaluated? AI: 0-100 score + feedback + suggestions. Rule-based: a fixed 4×25 rubric - relevance (question keywords addressed), evidence (numbers/examples/action verbs), technical coverage (vocabulary terms from the question), completeness (word-count bands) - plus one tip for the weakest dimension.

Q24. Describe the mock-interview rounds. Three rounds, 11 questions: Aptitude (5 MCQs, 15 min), Technical (3 free-text, 25 min), HR & Behavioral (3 free-text, 15 min). The session lives in server memory, questions are generated from the resume + JD, and each answer is graded instantly.

Q25. How is MCQ cheating prevented? The server strips `correctAnswerIndex` before sending questions to the browser and grades MCQs itself (10 or 0) when the answer is submitted - the answer key never reaches the client.

Q26. How does proctoring work? In the browser: TensorFlow.js with BlazeFace (face/gaze) and CocoSSD (objects) from a CDN, plus webcam. Gaze ratio outside [0.58, 1.7] or no face → GAZE_AWAY; a detected "cell phone" with score > 0.5 → PHONE_DETECTED; tab switches → TAB_SWITCH. Violations are posted to the backend.

Q27. What happens when violations exceed the limit? After the 3rd violation the session is marked TERMINATED (server + UI). A terminated session always scores 0 and its recommendation is "Not Recommended (Terminated for proctor violations)".

Q28. How is the final scorecard computed? Per-round percentages = round average ÷ (answered × 10); overall = mean of all answer scores × 10 (0 if none answered). Hiring bands: ≥80 Strong Hire, ≥65 Recommended, ≥50 Borderline, else Not Recommended; up to 4 strengths and improvement areas.

Q29. How does authentication work - and what are the security weaknesses? Honestly: it does not, server-side. Login is cosmetic and stores plaintext demo passwords in localStorage; there are no route guards, no tokens, and session IDs are guessable. What does exist: CORS allow-lists, upload type/size limits, server-side MCQ grading, secrets confined to `backend/.env`, sanitized error messages and content-free logs. Real auth is future work.

Q30. How is the project tested, and what are its main limitations? Frontend: 17 Vitest/jsdom suites; backend: 13 `node:test` suites with all AI calls stubbed - covered areas include fallback behaviour, CORS, PDF/OCR, question bank and

score formulas. Main limitations: no DB/persistence, no real auth, simulated speech-to-text, bypassable client-side proctoring, synthetic dataset, some demo-only UI areas and two unused dependencies (`recharts`, `zod`).

Appendix A - Items explicitly flagged as unverified / uncertain

1. ⚠ AI prompt (soft skills 10%) vs response schema/rule engine (education 10%) - documented inconsistency in `backend/llm.js` / `ruleBasedFallback.js`.
2. ⚠ `recharts` (frontend) and `zod` (backend) are declared in `package.json` files but no imports were found in application source.
3. ⚠ Root `README.md` is missing although `scripts/package-zip.ps1` requires it - packaging would currently fail; also several research documents referenced by `research/results/reproducibility/SHA256SUMS.txt` are absent from this checkout.
4. ⚠ Actual deployed `VITE_API_BASE_URL` and AI keys are environment-specific and were not inspected (no `.env` files read).
5. ⚠ `research/scripts/benchmark_corpus_v1.mjs` header says "NOT YET RUN" while its output files exist (stale comment v. artifacts) - do not cite generation details without re-checking.
6. ⚠ The manuscript `.docx` / `.pdf` files under `research/results/manuscript/` were not content- reviewed (binary files); treat their claims separately from this code-based guide.

Appendix B - Quick file map

Concern	File(s)
API routes	`backend/server.js`
Provider routing, caching, errors	`backend/aiProvider.js`
Match/questions/eval prompts + deterministic parser	`backend/llm.js`
Rule-based engine (scoring formulas)	`backend/ruleBasedFallback.js`
Curated question bank	`backend/questionBank.js`
PDF + OCR	`backend/pdfExtract.js`
Mock interview AI tasks	`backend/mockInterviewLlm.js`
In-memory interview sessions	`backend/mockInterviewStore.js`
Skill-gap / career-plan tasks	`backend/skillGapLlm.js`

Concern	File(s)
Frontend API client + errors	`src/lib/api-service.ts`, `src/lib/api-config.ts`
Client-side match "Career Fit" scorer	`src/lib/resume-analysis.ts`
Resume session store	`src/lib/resume-store.ts` (+ `src/main.tsx`)
Login (demo auth)	`src/pages/login.tsx`
Mock interview + proctoring UI	`src/pages/mock-interview.tsx`
Static job listings	`src/data/jobs.ts`
Routes	`src/routes/index.tsx`
Backend tests / fixtures	`backend/tests/`, `backend/tests/fixtures/`
Frontend tests	`src/**/*.test.ts(x)`
Synthetic dataset	`dataset/`
Research artifacts	`research/`